

HealthTrack Application – Technical Documentation

Arun Bhaskar Gadde

School of Computing and Information Sciences

University of the Cumberland

2026 Summer - Computer Science Capstone (MSCS-699-B01) - Second Bi-term

Michael Solomon PhD

August 09, 2026

HealthTrack Application – Technical Documentation

1. Project Structure

```

healthtrack_performance_optimization/
app/
  models.py          Patient, VitalsReading, RiskReport (SQLAlchemy)
  database.py        engine + session factory
  seed.py            generates 2,000 patients / 182,400 readings
  cache.py           TTLCache (in-memory, thread-safe)
  routes_v1_baseline.py unoptimized endpoints (the "before" state)
  routes_v2_optimized.py optimized endpoints (the "after" state)
  migrations/add_indexes.py
risk_engine/         reused risk-scoring module (see prior phase)
main.py             FastAPI app, mounts both v1 and v2 routers
benchmarks/
  bench_harness.py   sequential latency + SQL query counting
  bench_optimized.py runs the full before/after suite
  bench_cache.py     isolated cache hit vs. miss measurement
  bench_load.py      concurrent load test against a live server
  *_results.json     raw output from each script above
frontend_demo/
  before/           unminified dashboard, eager chart rendering
  after/            minified assets, lazy-loaded charts
docs/              this document and its companions

```

2. Why v1 and v2 Routes Both Exist

The unoptimized (v1) and optimized (v2) endpoints were kept side by side, mounted on the same running application against the same database, rather than optimizing routes in place and losing the "before" version. This was a deliberate choice: it allows every benchmark in this report to be re-run and independently verified at any time by anyone reviewing this work, rather than asking a reader to trust a one-time measurement. In a real migration, v1 would be deprecated and removed once v2 is validated in production; both are kept here purely for demonstration and reproducibility.

3. How Each Optimization Works

3.1 Composite index

Added via a plain SQL migration rather than an ORM-level index declaration, so it is explicit and reviewable: `CREATE INDEX ix_vitals_readings_patient_timestamp ON vitals_readings (patient_id, timestamp DESC)`. This single index serves both the `WHERE patient_id = ?` filter and the `ORDER BY timestamp DESC` clause used by every per-patient readings query, verified via `EXPLAIN QUERY PLAN` before and after (see Performance Analysis Report, Section 2).

3.2 N+1 elimination

The baseline dashboard endpoint queried all patients, then looped and issued one additional query per patient. The optimized version replaces the entire loop with a single SQLAlchemy query using `func.count`, `func.avg`, and `func.max` grouped by `patient_id`, executed once by the database engine itself. A second, separate batch query fetches patient names (not one per row) to attach to the aggregated results -- this keeps the fix at exactly 2 queries total regardless of how many patients exist, instead of $1 + N$.

3.3 Pagination and lightweight responses

List endpoints now accept `limit` and `offset` query parameters (default `limit=50`, capped at a maximum to prevent an accidental `limit=999999` from recreating the original problem) and return only the columns a list view actually needs, deferring full detail to per-resource endpoints. Total row counts are included in the response so a frontend can build pagination controls without a separate request.

3.4 In-memory TTL cache

Implemented as a small, dependency-free class (`app/cache.py`) rather than pulling in Redis for this phase (see Recommendations document for why Redis is the next step, not this one). The cache key is `(patient_id, latest_reading_id)`, which is important for correctness: the key itself changes the moment a new reading arrives for that patient, so the cache cannot silently serve a stale risk report -- there is no manual invalidation logic to get wrong. A 30-second TTL provides a safety net even in the (currently impossible, but worth defending against) case of a key collision or clock skew.

3.5 Bounded trend window

The baseline risk-report endpoint recomputed a trend score using the patient's entire reading history, meaning cost grew unboundedly with how long a patient had been enrolled -- exactly the pattern that made the 20 "heavy" seeded patients disproportionately expensive. The optimized version bounds this to the most recent 20 readings (`TREND_WINDOW` constant in `routes_v2_optimized.py`), which is clinically sufficient for a short-term trend signal and keeps the endpoint's cost constant regardless of enrollment length.

3.6 Frontend: minification and compression

`dashboard.js` and `dashboard.css` are minified with `rjsmin` / `rcssmin` (real, standard minifiers, not a hand-rolled regex) and additionally measured under gzip compression, since that is how they would actually be served in production behind any standard web server or CDN with compression enabled.

3.7 Frontend: lazy-loaded charts

The optimized dashboard.html defers five of its six chart widgets behind an IntersectionObserver, swapping a lightweight CSS skeleton placeholder for the actual <canvas> element and only calling the render routine once a section scrolls within 150px of the viewport. The first, above-the-fold chart still renders immediately, since deferring visible content would only hurt perceived load time.

4. Running This Project

Setup

```
pip install fastapi uvicorn sqlalchemy httpx psutil rjsmin rcssmin
python3 -m app.seed          # creates app/healthtrack_perf.db
python3 -m app.migrations.add_indexes # applies the index migration
```

Running the API

```
uvicorn app.main:app --reload
# v1 (baseline): http://127.0.0.1:8000/api/v1/...
# v2 (optimized): http://127.0.0.1:8000/api/v2/...
```

Reproducing the benchmarks

```
python3 benchmarks/bench_harness.py # baseline sequential latency
python3 benchmarks/bench_optimized.py # v1-with-index vs. v2 comparison
python3 benchmarks/bench_cache.py   # isolated cache hit/miss timing
python3 benchmarks/bench_load.py    # concurrent load test (starts its own server)
```

5. Testing Performed

Test type	Tool/Method	What it validates
Sequential latency	FastAPI TestClient, 20 repeated calls per endpoint, warmed up first	Per-request response time under no contention
SQL query counting	SQLAlchemy before_cursor_execute event hook	Proves the N+1 pattern was actually eliminated, not just "felt faster"
Query plan verification	SQLite EXPLAIN QUERY PLAN, before and after the index migration	Proves the index is actually used (SEARCH vs. SCAN), not just present
Concurrent load test	Live uvicorn server + httpx.AsyncClient + asyncio.gather	Throughput and tail latency under simultaneous requests, not just one at a time
Resource monitoring	psutil sampling of the live server process during load tests	CPU and memory behavior under load, not just response time
Cache correctness	Assertions on the X-Cache response header (HIT/MISS) during the cache benchmark	Confirms measured "hit" and "miss" timings are actually hitting the code path they claim to
Frontend asset size	Real minifiers (rjsmin/rcssmin) + Python's gzip module	Actual byte counts, not estimated compression ratios

All test scripts and their raw output are included in benchmarks/ so results can be independently reproduced rather than taken on faith.

6. Known Limitations of This Evaluation

- The original HealthTrack backend/dashboard codebase from earlier project phases was not present in this environment; the backend used here is a representative reconstruction matching the documented architecture (see Performance Analysis Report scope note), not the literal prior codebase.

- SQLite was used for the database layer for zero-setup portability in this environment; absolute latency numbers would differ on PostgreSQL, though the query patterns, index design, and N+1 fix are directly transferable (see Recommendations).
- The predictive ML component of the risk engine was not included in the request path benchmarked here (only rule-based scoring was), since it was already identified in the prior phase's validation documentation as a separate, heavier computation better suited to the background-job recommendation in this report.
- Load tests were run on a single machine against a co-located server process; true network latency to real clients is not represented in the throughput/latency figures.